```
"Content-Type": "application/json"
            },
            json=payload,
            timeout=config.REQUEST_TIMEOUT
        )

        if response.status_code >= 400:
            raise RuntimeError(
                f"Erro da API Groq ({response.status_code}): "
                f"{response.text[:400]}"
            )

        data = response.json()
        choices = data.get("choices", [])

        if not choices:
            raise RuntimeError(
                "O modelo não retornou uma resposta."
            )

        answer = choices[0]["message"]["content"]

        if not isinstance(answer, str) or not answer.strip():
            raise RuntimeError("Resposta vazia do
modelo.")

        return answer.strip()


#
==========================================
==================
# 5. AGENTE PRINCIPAL
#
==========================================
==================

class Viva:
```

```
def __init__(self):
    self.memory = memory
    self.cognition = CognitiveCore()
    self.language = LanguageEngine()

def respond(
    self,
    conversation_id: str,
    user_text: str
) -> tuple[str, int]:

    started = time.time()

    relevant_memories = self.memory.recall(user_text)
    lessons = self.memory.feedback_lessons()

    history = self.memory.history(conversation_id)

    system_prompt = self.cognition.build_prompt(
        user_text,
        relevant_memories,
        lessons
    )

    messages = [
        {
            "role": "system",
            "content": system_prompt
        }
    ]

    # Recupera a continuidade da conversa.
    messages.extend(history[-config.MAX_HISTORY:])

    answer = self.language.generate(messages)

    # Persiste a resposta antes de devolver ao usuário.
```

```
        message_id = self.memory.add_message(
            conversation_id,
            "assistant",
            answer
        )

        # Registra uma experiência resumida.
        self.memory.remember(
            "Pergunta: "
            + user_text[:400]
            + "\nResposta: "
            + answer[:700],
            kind="experience",
            confidence=0.45
        )

        self.memory.metric(
            "response_seconds",
            time.time() - started,
            {"model": config.MODEL}
        )

        return answer, message_id


five = Viva()


#
============================================
==================
# 6. INTERFACE WEB
#
============================================
==================

PAGE = r"""
<!DOCTYPE html>
```

```
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1">

<title>5 — Viva</title>

<style>
:root {
    color-scheme: dark;
    --bg: #101218;
    --panel: #191d27;
    --border: #303746;
    --text: #f0f2f7;
    --muted: #a9b1c2;
    --accent: #8ab4ff;
}

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    background: var(--bg);
    color: var(--text);
    font: 16px/1.5 system-ui, sans-serif;
}

main {
    max-width: 900px;
    margin: auto;
    min-height: 100dvh;
    display: flex;
    flex-direction: column;
    padding: 18px;
}
```

```
header {
  border-bottom: 1px solid var(--border);
  padding: 12px 0 18px;
}

h1 {
  margin: 0;
  font-size: 1.4rem;
}

header p,
#status,
footer {
  color: var(--muted);
}

#chat {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 14px;
  padding: 20px 0;
}

.message {
  padding: 14px;
  border: 1px solid var(--border);
  border-radius: 14px;
  white-space: pre-wrap;
  overflow-wrap: anywhere;
}

.user {
  align-self: flex-end;
  max-width: 90%;
  background: #202b40;
}
```

```
.assistant {
    background: var(--panel);
}

.role {
    font-size: .75rem;
    color: var(--muted);
    margin-bottom: 6px;
    text-transform: uppercase;
}

form {
    display: flex;
    gap: 10px;
    border-top: 1px solid var(--border);
    padding-top: 14px;
}

textarea {
    flex: 1;
    min-width: 0;
    min-height: 55px;
    max-height: 200px;
    padding: 12px;
    border-radius: 12px;
    border: 1px solid var(--border);
    background: var(--panel);
    color: var(--text);
    font: inherit;
}

button {
    cursor: pointer;
    border: 0;
    border-radius: 10px;
    padding: 10px 16px;
    background: var(--accent);
```

```
    color: #101218;
    font-weight: bold;
}

button:disabled {
    opacity: .5;
    cursor: wait;
}

.small {
    background: transparent;
    color: var(--muted);
    border: 1px solid var(--border);
    padding: 5px 9px;
    margin: 8px 5px 0 0;
}

footer {
    font-size: .8rem;
    padding-top: 12px;
}

@media (max-width: 600px) {
    main {
        padding: 12px;
    }
}
</style>
</head>

<body>
<main>

<header>
    <h1>5 — Viva</h1>
    <p>Memória, continuidade e reflexão
experimental.</p>
    <div id="status">Iniciando...</div>
```

```
</header>

<section id="chat"></section>

<form id="form">
  <textarea
    id="message"
    maxlength="12000"
    placeholder="Converse com o 5..."
    required
  ></textarea>
  <button id="send" type="submit">Enviar</button>
</form>

<footer>
  O feedback ajuda a orientar respostas futuras.
  Esta versão não modifica automaticamente os pesos
do modelo.
</footer>

</main>

<script>
let conversationId = null;

const chat = document.getElementById("chat");
const form = document.getElementById("form");
const input = document.getElementById("message");
const send = document.getElementById("send");
const statusLine = document.getElementById("status");

function addMessage(role, text, messageId = null) {
  const box = document.createElement("article");
  box.className = "message " + role;

  const label = document.createElement("div");
  label.className = "role";
```

```
    label.textContent = role === "user" ? "Você" : "5 — Viva";

    const body = document.createElement("div");
    body.textContent = text;

    box.append(label, body);

    if (role === "assistant" && messageId !== null) {
        const positive = document.createElement("button");
        positive.className = "small";
        positive.textContent = "Útil 👍";
        positive.onclick = () => feedback(messageId, 1);

        const negative = document.createElement("button");
        negative.className = "small";
        negative.textContent = "Melhorar 👎";
        negative.onclick = () => feedback(messageId, -1);

        box.append(positive, negative);
    }

    chat.appendChild(box);
    window.scrollTo({
        top: document.body.scrollHeight,
        behavior: "smooth"
    });
}

async function feedback(messageId, rating) {
    const comment = window.prompt(
        "O que deve ser mantido ou melhorado? (opcional)",
        ""
    ) || "";

    try {
```

```
        const response = await fetch("/api/feedback", {
            method: "POST",
            headers: {"Content-Type": "application/json"},
            body: JSON.stringify({
                conversation_id: conversationId,
                message_id: messageId,
                rating,
                comment
            })
        });

        const data = await response.json();

        if (!response.ok) {
            throw new Error(data.error || "Erro ao registrar
feedback.");
        }

        statusLine.textContent = "Feedback registrado.";
    } catch (error) {
        statusLine.textContent = error.message;
    }
}

async function startConversation() {
    try {
        const response = await fetch("/api/conversations", {
            method: "POST"
        });

        const data = await response.json();

        if (!response.ok) {
            throw new Error(data.error || "Erro ao iniciar.");
        }

        conversationId = data.conversation_id;
```

```
        statusLine.textContent = data.ai_ready
            ? "Modelo de linguagem disponível."
            : "Configure GROQ_API_KEY para conversar.";

        if (!data.ai_ready) {
            addMessage(
                "assistant",
                "A interface está pronta. Falta configurar a
chave da API Groq."
            );
        }
    } catch (error) {
        statusLine.textContent = error.message;
    }
}

form.addEventListener("submit", async event => {
    event.preventDefault();

    const message = input.value.trim();

    if (!message || !conversationId) {
        return;
    }

    input.value = "";
    addMessage("user", message);

    send.disabled = true;
    statusLine.textContent = "O 5 está processando...";

    try {
        const response = await fetch("/api/chat", {
            method: "POST",
            headers: {"Content-Type": "application/json"},
            body: JSON.stringify({
                conversation_id: conversationId,
                message
```

```
            })
        });

        const data = await response.json();

        if (!response.ok) {
            throw new Error(data.error || "Falha na
resposta.");
        }

        addMessage(
            "assistant",
            data.answer,
            data.message_id
        );

        statusLine.textContent = "Resposta recebida.";
    } catch (error) {
        addMessage("assistant", "Erro: " + error.message);
        statusLine.textContent = "Não foi possível
concluir.";
    } finally {
        send.disabled = false;
        input.focus();
    }
});

startConversation();
</script>
</body>
</html>
"""


#
==========================================
==================
# 7. API WEB
```

```
#
==========================================
==================

@app.get("/")
def home():
    return render_template_string(PAGE)


@app.get("/health")
def health():
    return jsonify({
        "status": "ok",
        "application": config.NAME,
        "version": config.VERSION,
        "model_configured": bool(config.API_KEY),
        "timestamp": now()
    })


@app.post("/api/conversations")
def create_conversation():
    try:
        conversation_id = memory.create_conversation()

        return jsonify({
            "conversation_id": conversation_id,
            "ai_ready": bool(config.API_KEY)
        })

    except Exception:
        app.logger.exception("Erro ao criar conversa")

        return jsonify({
            "error": "Não foi possível criar a conversa."
        }), 500
```

```
@app.post("/api/chat")
def chat():
    data = request.get_json(silent=True) or {}

    conversation_id = str(
        data.get("conversation_id", "")
    ).strip()

    user_text = str(
        data.get("message", "")
    ).strip()

    if not conversation_id or not user_text:
        return jsonify({
            "error": "Informe a conversa e a mensagem."
        }), 400

    if len(user_text) > config.MAX_MESSAGE_LENGTH:
        return jsonify({
            "error": "A mensagem excede o limite permitido."
        }), 413

    if not memory.conversation_exists(conversation_id):
        return jsonify({
            "error": "Conversa não encontrada."
        }), 404

    if not config.API_KEY:
        return jsonify({
            "error": "Configure GROQ_API_KEY no
ambiente."
        }), 503

    try:
        memory.add_message(
            conversation_id,
            "user",
            user_text
```

```
        )

        answer, message_id = five.respond(
            conversation_id,
            user_text
        )

        return jsonify({
            "answer": answer,
            "message_id": message_id,
            "conversation_id": conversation_id
        })

    except requests.RequestException:
        app.logger.exception("Falha de comunicação com
a API")

        return jsonify({
            "error": "Falha de conexão com o modelo de
linguagem."
        }), 502

    except Exception:
        app.logger.exception("Falha no núcleo do 5")

        return jsonify({
            "error": "Ocorreu um erro interno ao gerar a
resposta."
        }), 500


@app.post("/api/feedback")
def register_feedback():
    data = request.get_json(silent=True) or {}

    conversation_id = str(
        data.get("conversation_id", "")
    ).strip()
```

```
    try:
        rating = int(data.get("rating"))
    except (TypeError, ValueError):
        return jsonify({
            "error": "Avaliação inválida."
        }), 400

    if rating not in (-1, 0, 1):
        return jsonify({
            "error": "A avaliação deve ser -1, 0 ou 1."
        }), 400

    if not memory.conversation_exists(conversation_id):
        return jsonify({
            "error": "Conversa não encontrada."
        }), 404

    try:
        message_id = data.get("message_id")
        message_id = int(message_id) if message_id is not
None else None
    except (TypeError, ValueError):
        message_id = None

    comment = str(data.get("comment", "")).strip()[:1000]

    memory.record_feedback(
        conversation_id,
        message_id,
        rating,
        comment
    )

    return jsonify({
        "status": "recorded",
        "reflection_proposals": memory.reflect()
    })
```

```
@app.get("/api/memory")
def list_memories():
    # Endpoint de inspeção; não expõe o histórico
completo das conversas.
    with memory.connect() as db:
        rows = db.execute("""
            SELECT id, content, kind, confidence,
                   usefulness, created_at, last_used
            FROM memories
            ORDER BY id DESC
            LIMIT 100
        """).fetchall()

    return jsonify({
        "memories": [dict(row) for row in rows]
    })


@app.post("/api/memory")
def add_memory():
    data = request.get_json(silent=True) or {}

    content = str(data.get("content", "")).strip()
    kind = str(data.get("kind", "knowledge")).strip()

    if not content:
        return jsonify({
            "error": "O conteúdo da memória é obrigatório."
        }), 400

    if len(content) > 2500:
        return jsonify({
            "error": "Memória muito longa."
        }), 413

    memory_id = memory.remember(
```

```
        content,
        kind=kind[:50] or "knowledge",
        confidence=0.8
    )

    return jsonify({
        "status": "saved",
        "memory_id": memory_id
    })


@app.get("/api/reflections")
def list_reflections():
    with memory.connect() as db:
        rows = db.execute("""
            SELECT id, observation, proposal, status,
created_at
            FROM reflections
            ORDER BY id DESC
            LIMIT 100
        """).fetchall()

    return jsonify({
        "reflections": [dict(row) for row in rows]
    })


@app.get("/api/metrics")
def metrics():
    return jsonify(memory.statistics())


@app.errorhandler(404)
def not_found(error):
    if request.path.startswith("/api/"):
        return jsonify({
            "error": "Endpoint não encontrado."
        }), 404
```

```
    return "Página não encontrada.", 404


#
==========================================
==================
# 8. INICIALIZAÇÃO
#
==========================================
==================

if __name__ == "__main__":
    port = int(os.getenv("PORT", "5000"))

    app.run(
        host="0.0.0.0",
        port=port,
        debug=False
    )
```